Module 2 — Programming for AI & Data Science

Reading

> ***Role of this reading:*** *This reading is your canonical reference for Module 2. It defines every term precisely, documents every procedure step by step, and addresses the edge cases you will encounter in the lab and integration task. Read it before the concept session. Return to it as reference material throughout the week.*

Learning Outcomes

By the end of this module, you will be able to:

1. Build a modular, end-to-end data pipeline script using NumPy, Pandas, and Matplotlib that loads raw data, cleans it, transforms it, generates visualizations saved as files, and outputs summary statistics.
2. Apply modular coding practices — function decomposition, descriptive naming, `if __name__ == "__main__"` guard, separation of loading/processing/output — so that pipeline components can be tested and reused independently.
3. Write and run unit tests with pytest to validate individual pipeline functions using assertions, parametrized tests, and the `pytest` CLI.
4. Install PyTorch (CPU-only build) with understanding of why and how the CPU vs. GPU install decision is made, and use core PyTorch constructs — tensors, `nn.Module`, forward pass, loss functions, and an optimizer — to train a minimal tabular regression model.
5. Explain the architectural differences between PyTorch and TensorFlow at a conceptual level — eager vs. graph execution, where each framework dominates, how Keras relates to TensorFlow, and the factors that drive framework choice.

In This Reading

1. What Is a Data Pipeline and Why Does It Matter?
2. NumPy for Data Science
3. Pandas for Data Pipelines
4. Matplotlib for Visualization
5. Modular Coding Practices
6. Unit Testing with pytest
7. Reproducible Notebooks vs. Scripts
8. Installing PyTorch — The CPU vs. GPU Decision
9. PyTorch Fundamentals
10. Deep Learning Framework Landscape: PyTorch vs. TensorFlow
    - Terminology Reference

---

1. What Is a Data Pipeline and Why Does It Matter?

A **data pipeline** is a sequence of data processing steps that transforms raw input into useful output. The output might be a cleaned dataset, a set of charts, a summary report, or a feature matrix ready for a machine learning model. The defining characteristic of a pipeline is that it is **reproducible**: given the same input, it always produces the same output.

Pipeline Stages

Every data pipeline follows the same basic arc, even if the details vary by domain:

```
load → validate → clean → transform → analyze/visualize → output
```

Stage	What happens
Load	Read raw data from a file, database, or API into memory
Validate	Confirm the data has the expected shape, columns, and types
Clean	Handle missing values, fix malformed entries, correct types
Transform	Compute derived features, aggregate, reshape, join
Analyze / Visualize	Generate statistics, create charts
Output	Save processed data, charts, and reports to files

Why Modularity Matters

A monolithic pipeline — 200 lines of code with no functions — is fragile. When one step fails, you have to read the entire script to find the problem. When the data format changes, every assumption is scattered through the code.

A **modular pipeline** structures each stage as a separate function. This matters for three reasons:

1. **Testability.** You can write a test for `clean_data()` without running the entire pipeline. When the test fails, you know exactly where the bug is.
2. **Debuggability.** When `generate_summary()` produces wrong numbers, you can call it in isolation with known input and inspect the output.
3. **Reusability.** A well-written `load_data()` function works for any CSV with the same schema — you don't rewrite it every project.

The Pipeline as a Professional Standard

Every ML model, analytics report, and dashboard starts with a pipeline. The quality of your model depends on the quality of your data preparation. A model trained on improperly cleaned data will produce wrong predictions, and those errors are difficult to trace back to the root cause. Building reliable pipelines is one of the most consistently valued skills in data science and ML engineering roles.

---

2. NumPy for Data Science

NumPy is the foundation of numerical computing in Python. Pandas DataFrames are backed by NumPy arrays internally. PyTorch tensors share memory layout conventions with NumPy. Understanding NumPy gives you insight into how all of these tools work under the hood.

Array Creation

```python
import numpy as np

a = np.array([1, 2, 3, 4, 5])            # From a Python list
b = np.zeros((3, 4))                      # 3×4 array of zeros
c = np.ones((2, 3))                       # 2×3 array of ones
d = np.arange(0, 10, 2)                   # [0, 2, 4, 6, 8]
e = np.linspace(0, 1, 5)                  # 5 evenly spaced values from 0 to 1
f = np.random.randn(100)                  # 100 samples from standard normal dist
```

Indexing and Slicing

```python
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

arr[0]          # First row: [1, 2, 3]
arr[0, 1]       # Row 0, column 1: 2
arr[:, 1]       # All rows, column 1: [2, 5, 8]
arr[1:, :2]     # Rows 1+, columns 0-1: [[4, 5], [7, 8]]

# Boolean masking — select elements meeting a condition
arr[arr > 5]    # [6, 7, 8, 9]
```

Vectorized Operations

Vectorized operations apply computations to entire arrays without Python-level loops. This is orders of magnitude faster than looping because NumPy delegates to optimized C code.

```python
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

a + b           # [11, 22, 33, 44] — element-wise addition
a * b           # [10, 40, 90, 160]
a ** 2          # [1, 4, 9, 16]
np.sqrt(a)      # [1.0, 1.414, 1.732, 2.0]
np.log(b)       # Natural log of each element
np.exp(a)       # e^x for each element
```

Timing comparison: Summing 1 million numbers with a Python loop takes ~100ms. With `np.sum()`, it takes ~1ms — a 100× speedup for a trivial operation. This gap grows with dataset size and operation complexity.

Broadcasting

Broadcasting is NumPy's set of rules for operating on arrays of different shapes. NumPy automatically expands dimensions to make the shapes compatible.

```python
arr = np.array([[1, 2, 3], [4, 5, 6]])   # Shape: (2, 3)
row = np.array([10, 20, 30])              # Shape: (3,)

arr + row   # row is broadcast across both rows: [[11, 22, 33], [14, 25, 36]]
```

The key rule: dimensions are compatible if they are equal, or if one of them is 1. Broadcasting is powerful but can cause subtle bugs — always check `.shape` when results look wrong.

Aggregation

```python
arr = np.array([[1, 2, 3], [4, 5, 6]])

np.sum(arr)             # 21 — sum of all elements
np.mean(arr)            # 3.5
np.std(arr)             # Standard deviation
np.min(arr)             # 1
np.max(arr)             # 6

# With axis parameter:
np.sum(arr, axis=0)     # Sum each column: [5, 7, 9]
np.sum(arr, axis=1)     # Sum each row: [6, 15]
np.mean(arr, axis=0)    # Mean of each column: [2.5, 3.5, 4.5]
```

Relationship to Pandas

DataFrame columns are backed by NumPy arrays. You can extract the underlying array with `.values` or `.to_numpy()`:

```python
import pandas as pd

df = pd.DataFrame({'a': [1, 2, 3], 'b': [4, 5, 6]})
arr = df['a'].to_numpy()   # NumPy array: [1, 2, 3]
arr = df.values            # Full DataFrame as 2D NumPy array
```

---

3. Pandas for Data Pipelines

Pandas provides two primary data structures: **Series** (1D labeled array) and **DataFrame** (2D table with labeled rows and columns). DataFrames are the primary tool for data pipeline work.

Loading Data

```python
import pandas as pd

# Basic load
df = pd.read_csv('data/sales_records.csv')

# With options
df = pd.read_csv(
    'data/sales_records.csv',
    parse_dates=['date'],          # Parse the 'date' column as datetime
    dtype={'store_id': int},       # Specify column types
    na_values=['', 'N/A', 'n/a'], # Treat these as NaN
    encoding='utf-8'               # Specify encoding (important for non-ASCII te
)
```

Edge case — encoding errors: When a CSV contains Arabic or other non-ASCII text, always specify `encoding='utf-8'`. If that fails, try `encoding='utf-8-sig'` (handles BOM) or `encoding='latin-1'` as a last resort.

Inspecting Data

These commands should always run after loading a new dataset:

```python
df.head()           # First 5 rows
df.tail(3)          # Last 3 rows
df.shape            # (rows, columns)
df.columns          # Column names
df.dtypes           # Data type of each column
df.info()           # Column names, types, and non-null counts
df.describe()       # Summary statistics for numeric columns
df.isna().sum()     # Count of NaN values per column
```

Selecting and Filtering

```python
# Single column → Series
s = df['product_category']

# Multiple columns → DataFrame
subset = df[['date', 'quantity', 'unit_price']]

# Label-based selection
row = df.loc[0]                              # Row with label 0
cell = df.loc[0, 'quantity']                 # Row 0, column 'quantity'

# Position-based selection
row = df.iloc[0]                             # First row by position
cell = df.iloc[0, 2]                         # Row 0, column position 2

# Boolean indexing — filter rows
high_value = df[df['unit_price'] > 100]
electronics = df[df['product_category'] == 'Electronics']
combined = df[(df['store_id'] == 1) & (df['unit_price'] > 50)]
```

Missing Data

Missing values in Pandas are represented as `NaN` (Not a Number) for numeric columns and `NaT` (Not a Time) for datetime columns.

```python
# Detection
df.isna()                # Boolean DataFrame: True where values are NaN
df.isna().sum()          # Count of NaN per column
df.isna().sum().sum()    # Total NaN count across the entire DataFrame

# Removal — use cautiously; drops data
df.dropna()                          # Drop rows where ANY column is NaN
df.dropna(subset=['quantity'])       # Drop rows where 'quantity' is NaN only
df.dropna(how='all')                 # Drop rows where ALL values are NaN

# Filling — preserve data
df['quantity'] = df['quantity'].fillna(df['quantity'].median())
df['unit_price'] = df['unit_price'].fillna(df['unit_price'].mean())
df['payment_method'] = df['payment_method'].fillna('Unknown')
df['quantity'] = df['quantity'].fillna(method='ffill')    # Forward-fill
df['quantity'] = df['quantity'].fillna(method='bfill')    # Backward-fill
```

Important: `df['col'].fillna(...)` returns a new Series — it does not modify `df` in place unless you reassign:

```python
df['quantity'] = df['quantity'].fillna(df['quantity'].median())
```

Edge case — integer columns with NaN: When a column has integer semantics but contains NaN, Pandas converts it to `float64` (because NaN is a float). If you need integer semantics, use the nullable integer type:

```python
df['quantity'] = df['quantity'].astype(pd.Int64Dtype())
```

Edge case — `fillna` scope: `df.fillna(df.mean())` fills every numeric column with its own mean. This is rarely what you want. Prefer `df[col].fillna(df[col].mean())` to be explicit.

Type Conversions

```python
# Convert to datetime
df['date'] = pd.to_datetime(df['date'])
# With error handling (malformed dates become NaT):
df['date'] = pd.to_datetime(df['date'], errors='coerce')

# Convert to numeric
df['quantity'] = pd.to_numeric(df['quantity'], errors='coerce')  # Non-numeric →

# Convert to categorical or string
df['store_id'] = df['store_id'].astype(str)
df['product_category'] = df['product_category'].astype('category')
```

Edge case — `pd.to_datetime` failures: Dates in inconsistent formats (e.g., some rows `YYYY-MM-DD`, others `DD/MM/YYYY`) will cause errors. Use `errors='coerce'` to convert unparseable entries to `NaT`, then handle `NaT` values explicitly.

Derived Columns

```python
# Arithmetic on columns
df['revenue'] = df['quantity'] * df['unit_price']

# Using assign() — returns a new DataFrame with the added column
df = df.assign(
    revenue=df['quantity'] * df['unit_price'],
    day_of_week=df['date'].dt.day_name()
)

# Vectorized string operations (preferred over apply for string work)
df['category_upper'] = df['product_category'].str.upper()
```

Groupby and Aggregation

```python
# Total revenue by product category
revenue_by_category = df.groupby('product_category')['revenue'].sum()

# Multiple aggregations with named results
summary = df.groupby('store_id').agg(
    total_revenue=('revenue', 'sum'),
    avg_order_value=('revenue', 'mean'),
    order_count=('revenue', 'count')
)

# transform() — returns a Series the same length as the original DataFrame
df['store_avg'] = df.groupby('store_id')['revenue'].transform('mean')
```

Sorting

```python
df.sort_values('revenue', ascending=False)   # Sort by revenue, high to low
df.sort_values(['store_id', 'date'])         # Sort by multiple columns
```

Merging

```python
# SQL-style join
merged = pd.merge(df_orders, df_products, on='product_id', how='left')

# Stack DataFrames
combined = pd.concat([df_jan, df_feb, df_mar], ignore_index=True)
```

Detailed join semantics are covered in Module 3 (SQL context).

Saving Output

```python
df.to_csv('output/cleaned_data.csv', index=False)   # index=False omits the row i
df.to_json('output/summary.json', orient='records')
```

---

4. Matplotlib for Visualization

Matplotlib is the foundational Python visualization library. It produces charts saved as image files — making it the right tool for pipeline output.

The Figure and Axes Model

Matplotlib has two levels: the **Figure** (the overall canvas) and one or more **Axes** (individual plot areas within the figure).

```python
import matplotlib.pyplot as plt

# The preferred pattern — explicit figure and axes
fig, ax = plt.subplots()                    # Single plot
fig, axes = plt.subplots(2, 3)              # 2-row, 3-column grid of plots
fig, ax = plt.subplots(figsize=(10, 6))    # Custom size in inches
```

Why this pattern? `plt.plot()` (direct calls) works for quick exploration but becomes confusing with multiple subplots. The explicit `fig, ax` pattern gives you clear control over which Axes you're drawing on.

Common Plot Types

```python
# Line chart
ax.plot(df['date'], df['revenue'], color='steelblue', linewidth=1.5, label='Reven

# Bar chart
ax.bar(categories, values, color='steelblue', edgecolor='white')

# Horizontal bar chart
ax.barh(categories, values, color='steelblue')

# Scatter plot
ax.scatter(df['quantity'], df['unit_price'], alpha=0.6, s=50)

# Histogram
ax.hist(df['revenue'], bins=30, edgecolor='white', color='steelblue')
```

Customization

```python
ax.set_title('Revenue by Product Category', fontsize=14, fontweight='bold')
ax.set_xlabel('Product Category', fontsize=12)
ax.set_ylabel('Total Revenue (JOD)', fontsize=12)
ax.legend(loc='upper right')
ax.grid(True, alpha=0.3)
ax.tick_params(axis='x', rotation=45)
```

Saving Figures

```python
import os

os.makedirs('output', exist_ok=True)

# Save — must call savefig() BEFORE show() or close()
fig.savefig('output/revenue_by_category.png', dpi=150, bbox_inches='tight')
plt.close(fig)   # Free memory
```

Critical edge case — `plt.show()` in scripts: In a pipeline script, `plt.show()` blocks execution. Use `fig.savefig()` and never `plt.show()` in automated pipelines.

Critical edge case — blank saved figures: Calling `plt.show()` before `fig.savefig()` clears the figure — the saved file will be blank. Always save first.

Edge case — output directory missing: `fig.savefig('output/chart.png')` fails if `output/` does not exist. Use `os.makedirs('output', exist_ok=True)` at the top of your visualization function.

Styles

```python
plt.style.use('seaborn-v0_8-whitegrid')  # Clean grid style
```

Note: This is a matplotlib built-in style. The `seaborn-v0_8-whitegrid` style name is part of matplotlib's style system and does not require any additional packages. The visualization library it references is introduced in Module 4.

Subplots

```python
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 5))
# ... draw on each axes ...
fig.tight_layout()
fig.savefig('output/overview.png', dpi=150, bbox_inches='tight')
plt.close(fig)
```

Notebooks vs. Scripts for Visualization

Context	Use
Notebook	Exploration, EDA, visualizing intermediate results
Script	Automated pipelines, reproducible output

The notebook reproducibility problem: Notebook cells can be run in any order. Hidden state from deleted or unexecuted cells is a common source of bugs. A pipeline script runs top-to-bottom every time.

---

5. Modular Coding Practices

Modular coding means structuring your code as independent, testable, reusable functions. This is not a style preference — it is the difference between code that works once and code that works reliably.

Function Decomposition

```python
import os
import pandas as pd

def load_data(filepath):
    """Load sales records from CSV. Returns a DataFrame."""
    df = pd.read_csv(filepath)
    print(f"Loaded {len(df)} records from {filepath}")
    return df

def clean_data(df):
    """Handle missing values and fix data types. Returns a cleaned DataFrame."""
    df = df.copy()  # Never modify the input in place
    df['quantity'] = df['quantity'].fillna(df['quantity'].median())
    df['unit_price'] = df['unit_price'].fillna(df['unit_price'].median())
    df['date'] = pd.to_datetime(df['date'], errors='coerce')
    return df

def add_features(df):
    """Compute derived columns. Returns an enriched DataFrame."""
    df = df.copy()
    df['revenue'] = df['quantity'] * df['unit_price']
    df['day_of_week'] = df['date'].dt.day_name()
    return df

def generate_summary(df):
    """Compute and return a summary statistics dict."""
    top_category = df.groupby('product_category')['revenue'].sum().idxmax()
    return {
        'total_revenue': df['revenue'].sum(),
        'avg_order_value': df['revenue'].mean(),
        'top_category': top_category,
        'record_count': len(df)
    }

def create_visualizations(df, output_dir='output'):
    """Generate and save 3 charts as PNG files."""
    os.makedirs(output_dir, exist_ok=True)
    # ... chart code ...

def main():
    df = load_data('data/sales_records.csv')
    df = clean_data(df)
    df = add_features(df)
    summary = generate_summary(df)
    print(summary)
    create_visualizations(df)

if __name__ == "__main__":
    main()
```

The `if __name__ == "__main__"` Guard

When Python imports a module, it runs all top-level code in that file. Without the guard, `main()` runs every time any other file imports your pipeline — including your test file.

```python
# Without guard: every import runs the pipeline immediately — breaks testing
main()

# With guard: main() only runs when you execute python pipeline.py directly
if __name__ == "__main__":
    main()
```

`__name__` is a built-in variable that equals `"__main__"` when the file is run directly, and equals the module name (e.g., `"pipeline"`) when imported.

Separation of Concerns

Each function should know about one thing:

- `load_data()` knows about files
- `clean_data()` knows about data quality
- `add_features()` knows about domain logic
- `create_visualizations()` knows about charts

`clean_data()` should not hardcode file paths. `load_data()` should not know about visualization. When concerns are separated, each function can be understood, tested, and modified without reading the others.

Constants and Configuration

Define paths, column names, and parameters at the top of the file:

```python
DATA_PATH = 'data/sales_records.csv'
OUTPUT_DIR = 'output'
NUMERIC_COLUMNS = ['quantity', 'unit_price']
DATE_COLUMN = 'date'
```

Docstrings

A brief one-line docstring for each function:

```python
def clean_data(df):
    """Handle missing values and type conversions. Returns a cleaned DataFrame.""
    ...
```

---

6. Unit Testing with pytest

Unit testing verifies that individual functions produce expected outputs for known inputs. A test that runs in 1 second and tells you exactly which function is broken is more valuable than an hour of debugging a full pipeline run.

Test File Conventions

pytest discovers tests by looking for:

- Files named `test_*.py` or `*_test.py`
- Functions named `test_*` inside those files

```
project/
├── pipeline.py
└── tests/
    └── test_pipeline.py    # pytest discovers and runs this
```

Basic Test Structure

```python
import pandas as pd
import numpy as np
from pipeline import load_data, clean_data, add_features

def test_load_data_returns_dataframe():
    df = load_data('data/sales_records.csv')
    assert isinstance(df, pd.DataFrame), "load_data must return a DataFrame"
    assert len(df) > 0, "DataFrame must not be empty"
    assert 'quantity' in df.columns, "DataFrame must contain 'quantity' column"

def test_clean_data_no_nulls():
    df = load_data('data/sales_records.csv')
    cleaned = clean_data(df)
    assert cleaned['quantity'].isna().sum() == 0, "No NaN in quantity after clean
    assert cleaned['unit_price'].isna().sum() == 0, "No NaN in unit_price after c

def test_add_features_creates_revenue():
    df = load_data('data/sales_records.csv')
    df = clean_data(df)
    df = add_features(df)
    assert 'revenue' in df.columns, "add_features must create a 'revenue' column"
    expected = df['quantity'] * df['unit_price']
    pd.testing.assert_series_equal(df['revenue'], expected, check_names=False)
```

Assertions

```python
assert condition, "message shown when assertion fails"

# Pandas-specific comparisons
pd.testing.assert_frame_equal(result_df, expected_df)
pd.testing.assert_series_equal(result_series, expected)
```

Fixtures

A **fixture** provides reusable test data. Instead of loading the CSV in every test, you load it once:

```python
import pytest

@pytest.fixture
def sample_df():
    """A small test DataFrame — does not depend on the actual CSV."""
    return pd.DataFrame({
        'date': ['2024-01-15', '2024-01-16', '2024-01-17'],
        'store_id': [1, 2, 1],
        'product_category': ['Electronics', 'Audio', 'Computing'],
        'quantity': [2.0, np.nan, 5.0],
        'unit_price': [150.0, 75.0, np.nan],
        'payment_method': ['Cash', 'Credit Card', 'Mobile Payment']
    })

def test_clean_data_fills_quantity_nulls(sample_df):
    cleaned = clean_data(sample_df)
    assert cleaned['quantity'].isna().sum() == 0
```

Using small, constructed DataFrames in fixtures makes tests faster, more isolated, and independent of the data file's exact contents.

Parametrize

`@pytest.mark.parametrize` runs the same test with multiple sets of inputs:

```python
import pytest

@pytest.mark.parametrize("quantity,price,expected", [
    (10, 2.5, 25.0),
    (20, 3.0, 60.0),
    (0,  5.0, 0.0),
])
def test_revenue_calculation(quantity, price, expected):
    q = pd.Series([quantity])
    p = pd.Series([price])
    result = compute_revenue(q, p)
    assert result.iloc[0] == expected
```

Running Tests

```
pytest tests/ -v                # Run all tests, verbose output
pytest tests/test_pipeline.py # Run a specific file
pytest tests/test_pipeline.py::test_clean_data_no_nulls  # Run one test
```

Example output:

```
PASSED  tests/test_pipeline.py::test_load_data_returns_dataframe
PASSED  tests/test_pipeline.py::test_clean_data_no_nulls
FAILED  tests/test_pipeline.py::test_add_features_creates_revenue
  AssertionError: add_features must create a 'revenue' column
```

The Red-Green-Refactor Cycle

1. **Red:** Write a test that fails. The test describes the behavior you want.
2. **Green:** Write the minimum code to make the test pass.
3. **Refactor:** Clean up the code without breaking the test.

This cycle is how professional engineers move fast without breaking things. Each passing test is a permanent guarantee that the function works as expected.

Test-Driven Debugging

When a pipeline stage produces wrong output:

1. Write `test_buggy_function()` that asserts the correct behavior
2. Run it — it fails (red)
3. Fix the function
4. Run again — it passes (green)
5. The test now permanently documents the bug and its fix

---

7. Reproducible Notebooks vs. Scripts

	Notebook	Script
Best for	Exploration, EDA, storytelling	Automated pipelines, CI/CD, production
Execution	Interactive, any cell order	Linear, top to bottom
State	Hidden state persists between runs	Fresh state every run
Reproducibility	Depends on cell execution order	Guaranteed
Version control	Messy diffs (JSON metadata)	Clean diffs

Module 2 standard: Pipeline code lives in `.py` scripts. Jupyter notebooks are available for exploratory work but are not submitted as pipeline deliverables.

The extraction workflow: Explore in a notebook → identify the clean logic → extract it into a script → test the script with pytest.

---

8. Installing PyTorch — The CPU vs. GPU Decision

PyTorch is not installed with your Pre-Work packages. This is intentional — the install requires a deliberate hardware decision.

Why the Install Is Different

Standard packages are installed with `pip install package-name`. PyTorch has multiple builds:

- A **CPU-only build** (~200MB) — no CUDA dependency, works on any machine
- **CUDA-specific builds** (~2GB+) — require a compatible NVIDIA GPU and CUDA toolkit

CPU-Only Install Command

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
```

What `--index-url` does: Points pip at a PyTorch-maintained index containing only CPU-compiled wheels. Without this flag, pip may download from PyPI, which can include different builds or fail to find the right version.

Verifying the Install

```python
import torch
print(torch.__version__)                              # e.g., "2.2.0+cpu"
print("CUDA available:", torch.cuda.is_available()) # Should print: False
```

What Would Change with a GPU

```
# Example CUDA 12.1 install:
pip install torch --index-url https://download.pytorch.org/whl/cu121
```

The model code is identical. Two differences: `torch.cuda.is_available()` returns `True`, and you can move tensors and models to GPU: `tensor.to('cuda')`, `model.to('cuda')`.

Why CPU Is Sufficient for This Course

All Module 2 training examples use small data samples (hundreds to low thousands of rows). Training a 2-layer linear model on 200 rows for 100 epochs takes approximately 1–3 seconds on any modern CPU. GPU acceleration becomes relevant for large models and large training corpora — those scenarios are discussed in Modules 5–7.

The Professional Insight

In production, the compute decision is driven by dataset size, model complexity, latency requirements, and budget — not by a default install script. Making this decision explicitly is an engineering habit.

---

9. PyTorch Fundamentals

Tensors

A **tensor** is a multi-dimensional array that supports automatic differentiation and can be moved to a GPU. For most purposes, tensors work exactly like NumPy arrays.

```python
import torch

# Creating tensors
a = torch.tensor([1.0, 2.0, 3.0])               # From a Python list
b = torch.zeros(3, 4)                            # 3×4 tensor of zeros
c = torch.ones(2, 3)                             # 2×3 tensor of ones
d = torch.randn(5, 3)                            # Random normal, shape (5, 3)

# Tensor properties
print(a.shape)     # torch.Size([3])
print(a.dtype)     # torch.float32
print(a.device)    # cpu
```

Converting between NumPy and PyTorch:

```python
import numpy as np

np_array = np.array([1.0, 2.0, 3.0])
tensor = torch.from_numpy(np_array)         # NumPy → Tensor (shares memory)
back_to_numpy = tensor.numpy()              # Tensor → NumPy (shares memory)
```

Note: `torch.from_numpy()` and `.numpy()` share memory — changes to one affect the other.

Operations:

```python
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])

a + b                         # Element-wise: [5, 7, 9]
a * b                         # Element-wise: [4, 10, 18]
torch.matmul(a, b)            # Dot product: 32.0

# Reshaping
x = torch.randn(12)
x.reshape(3, 4)               # Shape (3, 4)
```

Edge case — dtype mismatch: `torch.tensor(df.values)` inherits the NumPy dtype — often `float64`. PyTorch model weights default to `float32`. Always specify the dtype:

```python
X = torch.tensor(df.values, dtype=torch.float32)
```

Autograd — Automatic Differentiation

Training a neural network requires computing the **gradient** of the loss with respect to each model parameter. PyTorch computes these gradients automatically through **autograd**.

```python
x = torch.tensor(2.0, requires_grad=True)
y = x ** 2
y.backward()
print(x.grad)    # 4.0 — the gradient dy/dx at x=2
```

For neural network training, you do not compute gradients by hand. PyTorch tracks all operations on model parameters (which have `requires_grad=True` by default) and computes gradients automatically when you call `loss.backward()`.

Edge case — forgetting `zero_grad()`: Gradients in PyTorch accumulate by default. If you do not call `optimizer.zero_grad()` at each iteration, gradients from previous iterations add to current gradients. This causes incorrect weight updates and typically results in training divergence.

`nn.Module` — The Model Class

Every PyTorch model is a Python class that inherits from `torch.nn.Module`. You implement two methods:

- `__init__`: define the layers as attributes
- `forward`: define how data flows through the layers

```python
import torch.nn as nn

class SimpleRegressor(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.layer1 = nn.Linear(input_dim, 32)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(32, 1)

    def forward(self, x):
        x = self.relu(self.layer1(x))
        return self.layer2(x)

model = SimpleRegressor(input_dim=5)
```

Calling the model: `predictions = model(input_tensor)` automatically calls `forward()`. Never call `model.forward(input_tensor)` directly.

`nn.Linear(in_features, out_features)`: A fully-connected layer. Computes `output = input @ weight.T + bias`, where `weight` and `bias` are learnable parameters.

`nn.ReLU()`: Rectified Linear Unit activation. Output = `max(0, x)`. Introduces non-linearity so the model can learn patterns a purely linear function cannot represent.

Loss Functions

A **loss function** measures how wrong the model's predictions are.

```python
criterion = nn.MSELoss()    # Mean Squared Error — standard for regression

predictions = model(X_train)             # Shape: (N, 1)
loss = criterion(predictions, y_train)  # Scalar value

print(f"Loss: {loss.item():.4f}")    # .item() extracts a Python float
```

Edge case — shape mismatch: Predictions are often `(N, 1)` but targets may be `(N,)`. Fix with:

```python
y_train = y_train.reshape(-1, 1)    # Shape (N, 1)
```

Edge case — non-scalar backward: `loss.backward()` requires a scalar loss. `nn.MSELoss()` defaults to `reduction='mean'`, producing a scalar. If you override this, ensure you reduce the loss to a scalar before calling `backward()`.

Optimizers

An **optimizer** adjusts model weights based on computed gradients.

```python
# Stochastic Gradient Descent
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

# Adam — adaptive learning rate, generally preferred for beginners
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

`model.parameters()` returns all learnable parameters (weights and biases) in the model.

Adam vs. SGD: Adam adapts the learning rate for each parameter individually. For most problems, Adam converges faster and is less sensitive to the initial learning rate. SGD is still widely used — especially with momentum — but Adam is the standard starting point.

The Training Loop

The training loop repeats four steps for a specified number of **epochs** (complete passes through the training data):

```python
num_epochs = 100

for epoch in range(num_epochs):
    # Step 1: Forward pass
    predictions = model(X_train)

    # Step 2: Compute loss
    loss = criterion(predictions, y_train)

    # Step 3: Reset gradients — MUST happen before backward
    optimizer.zero_grad()

    # Step 4: Backward pass — compute gradients
    loss.backward()

    # Step 5: Update weights
    optimizer.step()

    if epoch % 10 == 0:
        print(f"Epoch {epoch:3d}: Loss = {loss.item():.4f}")
```

The four-step rhythm: Forward → Loss → Backward → Step. This structure does not change regardless of model complexity. Every PyTorch training script follows this same pattern.

Data Preparation for PyTorch

```python
import pandas as pd
import torch

df = pd.read_csv('data/housing.csv')

# 1. Separate features and target
X = df.drop(columns=['price_jod'])
y = df[['price_jod']]

# 2. Feature scaling — standardize to zero mean, unit variance
X_mean = X.mean()
X_std = X.std()
X_scaled = (X - X_mean) / X_std

# 3. Convert to PyTorch tensors
X_tensor = torch.tensor(X_scaled.values, dtype=torch.float32)
y_tensor = torch.tensor(y.values, dtype=torch.float32)

print(X_tensor.shape)   # torch.Size([200, 5])
print(y_tensor.shape)   # torch.Size([200, 1])
```

Why feature scaling matters: If one feature ranges 50–250 and another 30,000–150,000, the large-scale feature dominates gradient updates. Standardization ensures all features contribute equally, leading to faster and more stable convergence.

---

10. Deep Learning Framework Landscape: PyTorch vs. TensorFlow

Understanding both major frameworks — even if you code in only one — is a professional expectation.

Historical Context

	TensorFlow	PyTorch
Origin	Google Brain, 2015	Facebook / Meta AI Research, 2016
Early adoption	Dominated industry; Google infrastructure, TF Serving, TFLite	Rapid academic adoption; overtook TF in research papers by ~2019
Today	Strong in production deployment; mobile/edge	Dominant in research; default for Hugging Face

The Core Architectural Difference: Execution Model

TensorFlow 1.x — Static Computation Graph (define-and-run):

1. Define the entire computation graph as a data structure
2. Start a Session
3. Feed data and execute the graph inside the Session

You could not inspect intermediate values with Python `print()`. Debugging required specialized tools.

PyTorch — Dynamic Computation Graph (define-by-run): Each Python line executes immediately. The graph is built on the fly. You can print, inspect, and set breakpoints anywhere. PyTorch works exactly like normal Python code.

TensorFlow 2.x: Google made eager execution the default in TF 2.0. `tf.function` is a decorator that compiles Python functions into optimized TF graphs for production performance. TF 2.x closed much of the ergonomics gap, but the deployment tooling remains distinct.

Keras

Keras is the official high-level API for TensorFlow: `tf.keras`. It provides simpler model-building syntax:

```python
# Keras Sequential model (TensorFlow)
model = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation='relu', input_shape=(5,)),
    tf.keras.layers.Dense(1)
])
model.compile(optimizer='adam', loss='mse')
model.fit(X_train, y_train, epochs=100)
```

Compare to PyTorch `nn.Module`, which gives you more control at the cost of more explicit code.

Where Each Framework Is Strongest Today

Context	Preferred Framework
Research and academic publications	PyTorch
Hugging Face ecosystem	PyTorch (default backend)
Rapid prototyping	PyTorch
Production deployment (TF Serving)	TensorFlow
Mobile and edge (TFLite)	TensorFlow
Google Cloud ML	TensorFlow

Job Market Context

Both frameworks appear in job postings. Employers generally expect:

- Architectural understanding of both frameworks
- Deep coding fluency in at least one
- Ability to learn the other if the role requires it

Why This Course Standardizes on PyTorch

1. **Hugging Face** (Modules 6–7) uses PyTorch as the default backend for its transformer-based model libraries
2. **Dynamic graphs** are more intuitive for learners
3. **Research community momentum** — most new architectures appear in PyTorch first
4. **CPU-friendly** — efficient builds for small examples

Terminology Reference

Term	Definition
autograd	PyTorch's automatic differentiation engine
backward pass	Computing gradients via the chain rule, triggered by `loss.backward()`
batch	A subset of training data processed together; this module uses full-batch for simplicity
broadcasting	NumPy/PyTorch rules for operating on arrays of different shapes
CPU vs. GPU	CPU handles general computation; GPU parallelizes matrix operations for faster neural network training
data pipeline	A sequence of data processing steps transforming raw input into useful output
DataFrame	Pandas' primary 2D data structure; rows and labeled columns
derived feature	A new column computed from existing columns (e.g., `revenue = quantity × price`)
dynamic computation graph	PyTorch's approach: graph built on each forward pass, enabling natural Python debugging
eager execution	Executing operations immediately as called (PyTorch default; TF 2.x default)
epoch	One complete pass through the training data
feature scaling	Normalizing input features (e.g., subtract mean, divide by std)
figure	Matplotlib's top-level container for all plot elements
fixture (pytest)	A function decorated with `@pytest.fixture` providing reusable test data
forward pass	Running input data through the model to produce predictions
graph execution	TensorFlow 1.x approach: define the computation graph, then execute it
`if __name__ == "__main__"`	Python guard that runs code only when the script is executed directly
imputation	Replacing missing values with estimated values
index URL (pip)	A URL pointing to a package repository; `--index-url` overrides the default PyPI
Keras	High-level neural network API, now integrated into TensorFlow as `tf.keras`
learning rate	Hyperparameter controlling how much the optimizer adjusts weights on each step
loss function	Measures how wrong the model's predictions are; training minimizes this
modular coding	Structuring code as independent, testable, reusable functions
`nn.Module`	PyTorch base class for all neural network models
optimizer	An algorithm updating model weights based on gradients (e.g., SGD, Adam)
parametrize (pytest)	Decorator that runs a test function with multiple sets of inputs
red-green-refactor	Test-driven development cycle: failing test → make it pass → clean up
Series	Pandas' 1D labeled array; a single DataFrame column
static computation graph	TensorFlow 1.x approach: full computation defined before execution
tensor	Multi-dimensional array with automatic differentiation support
training loop	Repeated cycle of forward pass → loss → backward pass → optimizer step
vectorized operation	Operation applied to entire arrays/Series at once, avoiding Python-level loops

---

Module 2 of 12 — Applied AI & ML Systems

© 2026 LevelUp Economy. All rights reserved. Unauthorized reproduction or distribution of this material is prohibited.